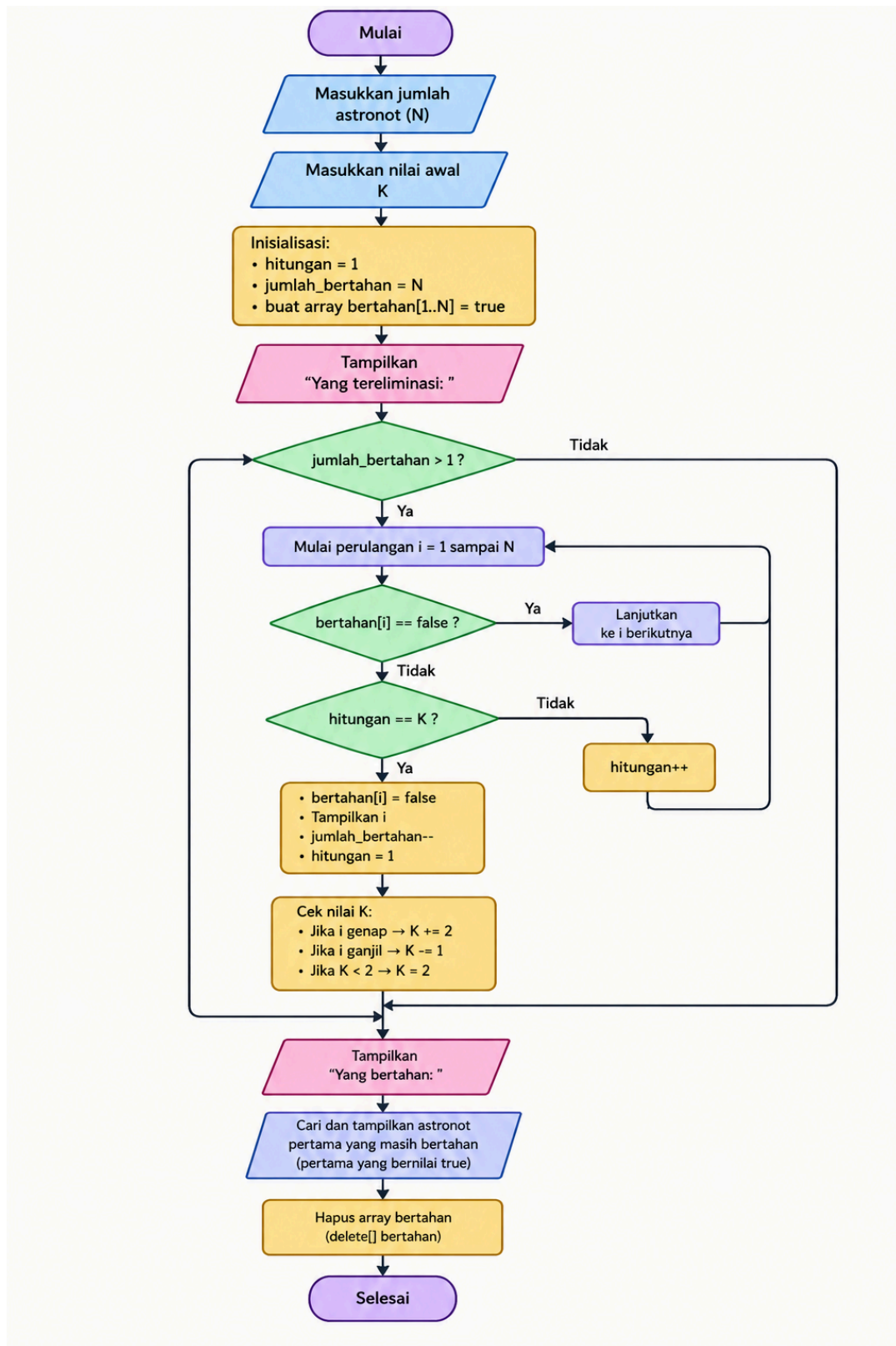


Nama : Arya Faeyza Herdi Dewanto

Kel : Onyx

Problem 1 : The Last Astronaut



Algoritma ini dirancang untuk mensimulasikan proses seleksi eliminasi astronot yang posisinya membentuk lingkaran. Pada tahap awal, program akan meminta pengguna memasukkan jumlah total astronot (N) serta nilai awal eliminasi (K). Setelah data diterima, sistem menyiapkan sebuah array yang berfungsi sebagai penanda bahwa seluruh astronot masih berstatus bertahan atau hidup. Beberapa variabel pendukung juga ikut diinisialisasi, seperti angka hitungan yang dimulai dari 1 dan jumlah astronot bertahan yang diset sama dengan N. Seluruh persiapan ini nantinya menjadi pondasi untuk perulangan utama yang akan terus berputar selama masih ada lebih dari satu astronot yang tersisa.

Saat masuk ke dalam perulangan, program mengecek status tiap astronot satu per satu secara berurutan. Kalau astronot di posisi tertentu sudah tereliminasi, sistem akan langsung melewatkannya dan lanjut ke orang berikutnya. Sebaliknya, jika astronot masih bertahan, angka hitungan akan naik terus sampai nilainya pas dengan target K. Begitu hitungan mencapai angka K, astronot tersebut resmi dinyatakan gugur, nomor urutnya dicetak ke layar, dan hitungan direset kembali ke angka 1. Bersamaan dengan itu, nilai K juga ikut diperbarui secara dinamis: ditambah 2 jika nomor astronot yang gugur itu genap, atau dikurangi 1 jika ganjil, asalkan hasil akhirnya tidak kurang dari 2. Proses memutar ini terjadi terus-menerus hingga menyisakan satu pemenang saja, di mana program akan mencetak nomor astronot terakhir tersebut sebelum akhirnya membersihkan sisa memori.

Problem 2 : Alien-In-The-Middle

```
START FUNCTION charToHuruf(ch)
  IF ch >= 'a' AND ch <= 'z' THEN
    RETURN ASCII(ch) - ASCII('a') + 1
  ELSE
    RETURN ASCII(ch) - ASCII('A') + 1
  END IF
END FUNCTION

START FUNCTION hurufToChar(huruf, lowercase)
  IF lowercase == TRUE THEN
    RETURN karakter dari (huruf + ASCII('a') - 1)
  ELSE
    RETURN karakter dari (huruf + ASCII('A') - 1)
  END IF
END FUNCTION

START FUNCTION hitungPanjang(arr)
  SET panjang = 0
  WHILE arr[panjang] != '\0'
    SET panjang = panjang + 1
  END WHILE
  RETURN panjang
END FUNCTION
```

```

START
  DECLARE array pesan[1000]
  PRINT "Masukkan pesan: "
  INPUT pesan

  SET panjang = CALL hitungPanjang(pesan)

  DECLARE array sandi[1000]
  SET prevChar = 0

  FOR i = 0 TO panjang - 1
    SET ch = CALL charToHuruf(pesan[i])
    SET ch = ch + prevChar

    WHILE ch > 26
      SET ch = ch - 26
    END WHILE

    IF pesan[i] >= 'a' AND pesan[i] <= 'z' THEN
      SET is_lowercase = TRUE
    ELSE
      SET is_lowercase = FALSE
    END IF

    SET sandi[i] = CALL hurufToChar(ch, is_lowercase)
    SET prevChar = CALL charToHuruf(pesan[i])
  END FOR

  SET sandi[panjang] = '\0'

  PRINT "Pesan sandi: " + sandi
END

```

Algoritma ini pada dasarnya bekerja dengan mengotak-atik posisi huruf di dalam susunan alfabet. Sebelum mulai ke inti program, kita perlu nyiapin tiga fungsi pendukung. Fungsinya masing-masing dipakai buat ngitung total huruf secara manual, mengubah karakter huruf jadi angka dari 1 sampai 26, dan nanti ngerubah angkanya balik lagi jadi bentuk huruf. Saat program pertama kali dijalankan, langkah awalnya bakal minta kita buat ngetik pesan teks. Pas pesannya sudah masuk dan panjangnya ketemu, program langsung nyiapin tempat penampungan berupa array bernama sandi. Barengan sama itu, dibuat juga variabel prevChar yang dikasih nilai awal 0. Variabel ini punya tugas krusial buat nyimpen angka dari huruf sebelumnya supaya bisa dipakai sebagai patokan geser buat huruf sesudahnya.

Kalau semua persiapan tadi udah beres, proses enkripsinya langsung jalan pakai sistem perulangan buat ngecek tiap karakter dari awal sampai akhir kalimat. Pola kerjanya sebenarnya cukup runtut. Tiap huruf yang lagi dibaca bakal diubah dulu jadi angka, terus ditambahin sama nilai prevChar. Cuma kadang, hasil penjumlahan itu bisa ngelewatin angka

26. Makanya, kalau kelebihan, angkanya bakal otomatis dikurangi 26 berulang kali sampai posisinya muter balik pas di rentang alfabet. Habis angka barunya ketemu, nilainya dibalikin lagi jadi huruf dengan tetap mengikuti bentuk aslinya apakah huruf besar atau kecil lalu disimpan ke penampungan sandi. Sebelum loncat ke huruf berikutnya, prevChar langsung di-update pakai angka dari huruf asli yang barusan di cek. Perulangan ini bakal terus berputar sampai seluruh huruf di kalimat habis, ditutup dengan tanda batas string di ujung array supaya pesannya bisa tampil utuh di layar.